ENTERPRISE INFORMATION SYSTEMS

EIS Implementation & Project Management

Weeks 07 & 08 · Business Information Systems

Cyber.SoHo Educational Hub · Montreal

WEEKS 07 & 08

What you'll be able to do today

- Choose an EIS go-live method (Big Bang, Phased, Parallel, Pilot) for a real situation
- Build and read a risk register, and name the critical success factors
- Explain how EIS modules share one database and mirror the organisation
- Trace a business process (Order-to-Cash) across several modules
- Compare EIS provisioning options and judge them on 5-year cost (TCO)

Today's map

- Lesson 1 - EIS Implementation
- Lesson 2 - EIS Project Management
- 2 short videos (<=15 min each)
- Live code demos + a 6-question quiz

OUR ANCHOR COMPANY

Meet Northern Roast Coffee Co.

- A small coffee roaster: one roastery plus three cafes (about 40 staff)
- Runs today on spreadsheets, a shoebox of invoices, and one tired office manager (Dana)
- Decision made: buy a proper EIS to tie the whole business together
- Fictional company - but the same problems whether you sell espresso, insurance, or aircraft parts

Why an anchor?

- Every idea today attaches to Northern Roast, so it never floats off into abstraction.
- When you see a concept, picture Dana trying to use it on Monday morning.

VOCABULARY

Two words you'll hear all day: EIS & ERP

- EIS (Enterprise Information System) = the wide umbrella term
- ERP (Enterprise Resource Planning) = the most common kind you would actually buy
- Same picture either way: one system, many departments, shared data

Market names you'll meet today (we sell none of them):

SAP

Oracle NetSuite

Microsoft Dynamics 365

Salesforce

Workday

The mental model

Type a fact once - a customer, a price, a shipment - and every part of the company sees it instantly.

That single shared picture is the whole point of an enterprise system.

Official links for every vendor are in the appendix.

CYBER.SOHO

Lesson 1 - EIS Implementation

Getting the system in: methods, risks, and what makes projects succeed

PART 1.1

Methods & approaches to EIS implementations

- Buying the software is easy - implementation is everything that happens after
- Loading data, wiring it to how the company works, training people, the nervous go-live moment
- The Standish Group (CHAOS research) keeps finding many big projects miss budget, deadline, or goals
- There are four classic ways to go live - and choosing between them is a top decision

The question

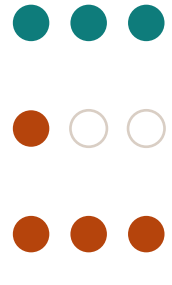
- How, exactly, do you switch the old system OFF and the new one ON?
- You don't just flip one giant switch and hope.

GO-LIVE METHOD 1

Big Bang

- Everyone switches to the new system on a single date
- On Friday you're on spreadsheets; on Monday the whole company is on the EIS - old system gone
- Fastest and cheapest: you never pay to run two systems at once
- Scariest: if Monday breaks, everything breaks at once - no safety net

AT A GLANCE



more dots = more of that trait

Best for

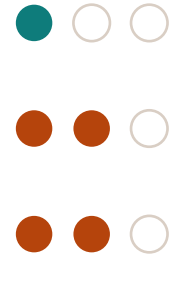
A small, single-site company with a tight deadline and staff ready for change.

GO-LIVE METHOD 2

Phased

- Turn the new system on one piece at a time - finance, then sales, then inventory
- Each step is smaller and easier to recover from; people learn gradually
- While some parts are new and some old, you build temporary bridges using written SOPs
- (SOP = Standard Operating Procedure.) Calmer overall, but takes longer and costs more

AT A GLANCE



more dots = more of that trait

Best for

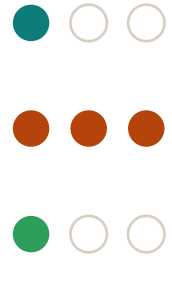
A company that wants a calm transition and can wait a little longer.

GO-LIVE METHOD 3

Parallel

- Run the old system AND the new system side by side, doing every task twice
- Switch the old one off only once you completely trust the new one
- Safest of all - the old system is right there to catch any new mistake
- Most exhausting and expensive; the right call for payroll or financial data

AT A GLANCE



more dots = more of that trait

Best for

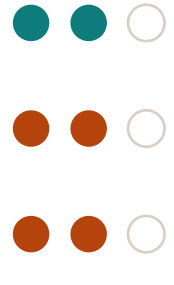
Data too important to risk - where a wrong number is unacceptable.

GO-LIVE METHOD 4

Pilot

- Go live at just one location or team first - for Northern Roast, the smallest cafe
- Only after it works there do you roll it out everywhere else
- Learn on a small, contained group before betting the whole company
- Middle of the road on speed, cost and risk; shines when you have many sites

AT A GLANCE



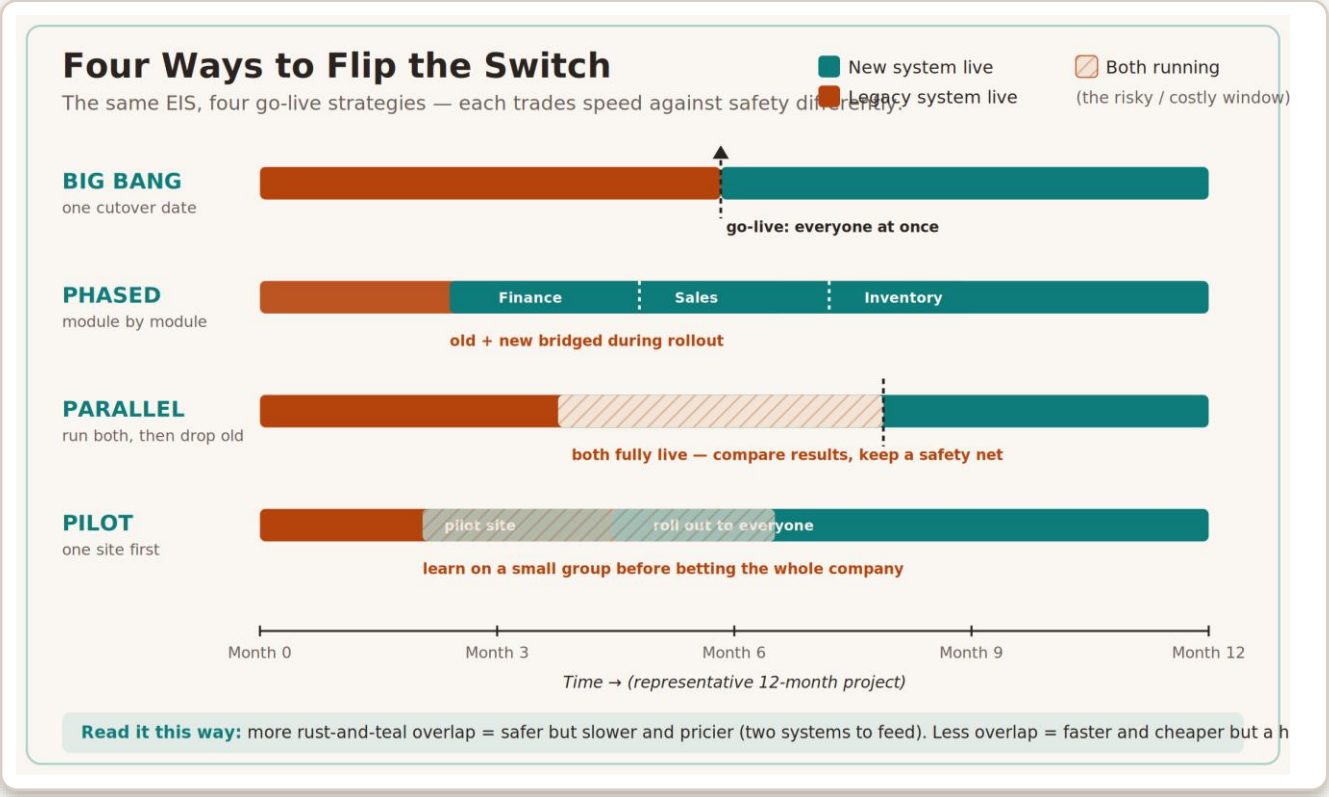
more dots = more of that trait

Best for

An organisation with many sites that wants to prove it once first.

THE ONE RULE BEHIND ALL FOUR

Figure 1 - Four ways to flip the switch



More overlap of old & new = more safety, but more time & cost.

Rust = old system live · Teal = new system live · Hatched = both running (the danger + cost window). Most firms mix: Big Bang the core finance, Phase in the rest - that's normal.

REAL-LIFE EXAMPLE

Dana's dilemma at Northern Roast

What it shows & how it links: the 'best' method is never universal - it depends on the company's size, deadline and appetite for risk (Part 1.1).

- The facts: about 40 staff across 4 sites; the old system wastes roughly $3,000/month; the owners say plainly 'we cannot afford a disaster during business hours'; and there is no hard deadline.
- Big Bang is out - low risk tolerance rules out a hard landing with no safety net.
- No deadline, so speed is not the priority; and the data (cafe takings, supplier payments) is too important to risk.
- Decision: run finance in PARALLEL until the numbers are trusted, then PHASE in the rest, cafe by cafe.

CODE ILLUSTRATION

Rollout strategy simulator (1/2)

```
# Describe the company we are rolling the system out to (change these to explore).
# -- how many people will use the system
NORTHERN_ROAST_USERS = 40
# -- how many physical locations (roastery + cafes); more sites = more moving parts
NORTHERN_ROAST_SITES = 4
# -- how much cost, per month, to keep the OLD system alive while we transition
OLD_SYSTEM_MONTHLY_COST = 3000
# -- the company's appetite for risk: "low", "medium" or "high"
RISK_TOLERANCE = "low"
# -- is there a hard deadline pushing us to go fast? True or False
UNDER_DEADLINE = False

# Define each strategy's "personality" as a dictionary of typical characteristics.
# base_months = rough length of the project; overlap_months = time BOTH systems run;
# risk_1to5 = inherent riskiness (5 = scariest single landing).
STRATEGIES = {
    # Big Bang: one date, everyone switches; short but a hard landing, no safety net.
    "Big Bang": {"base_months": 6, "overlap_months": 0, "risk_1to5": 5},
    # Phased: module by module; longer, with a bridge period where old + new coexist.
    "Phased": {"base_months": 11, "overlap_months": 5, "risk_1to5": 3},
    # Parallel: run old and new together, then drop old; safest but most expensive.
    "Parallel": {"base_months": 9, "overlap_months": 4, "risk_1to5": 2},
    # Pilot: prove it at one site first, then roll out; middle-of-the-road on all counts.
    "Pilot": {"base_months": 8, "overlap_months": 3, "risk_1to5": 3},
}

# Make a helper that scores one strategy and returns its numbers as a dictionary.
def evaluate(name, spec):
    # Duration is just the base length of that approach, in months.
    duration = spec["base_months"]
    # The extra cost of overlap = months of double-running x the old system's monthly cost.
    double_run_cost = spec["overlap_months"] * OLD_SYSTEM_MONTHLY_COST
    # Risk grows a little with each extra site, because there is more to coordinate.
    site_factor = 1 + (NORTHERN_ROAST_SITES - 1) * 0.1
    # Combine the inherent risk with the site factor to get a final risk score.
    risk_score = round(spec["risk_1to5"] * site_factor, 1)
    # Hand back all three measures plus the name, bundled together.
    return {"name": name, "months": duration, "double_run_cost": double_run_cost, "risk": risk_score}
```

What it does

- Scores all four methods on time, cost and risk for Northern Roast, then recommends one
- Links to Part 1.1: it turns the speed-vs-safety trade-off into numbers you can change

Reading the panel

Muted lines are comments (one above every line of code); light lines are the code itself.

CODE ILLUSTRATION

Rollout strategy simulator (2/2)

```
# Run every strategy through the helper and collect the results in a list.
results = [evaluate(name, spec) for name, spec in STRATEGIES.items()]

# Print a title so the output is easy to read.
print("Northern Roast Coffee Co. -- EIS rollout options\n")
# Print a header row for our comparison table, lined up in fixed-width columns.
print(f"{'Strategy':<12}{'Months':>8}{'Double-run $':>15}{'Risk (1-5)':>12}")
# Print a divider line under the header.
print("-" * 47)
# Loop through each result and print one neat row per strategy.
for r in results:
    # Format the money with a thousands separator and a dollar sign for readability.
    print(f"{r['name']:<12}{r['months']:>8}{('$'+format(r['double_run_cost'], ',')):>15}{r['risk']:>12}")

# Now turn the numbers into advice, using the company's risk tolerance and deadline.
# If the firm cannot stomach risk, favour the safest approach (lowest risk score).
if RISK_TOLERANCE == "low":
    # Pick the strategy with the smallest risk value from our results list.
    pick = min(results, key=lambda r: r["risk"])
    # Explain the reasoning behind the choice.
    reason = "you told me risk tolerance is LOW, so safety beats speed"
# Otherwise, if there is a hard deadline, favour the fastest approach (fewest months).
elif UNDER_DEADLINE:
    # Pick the strategy that finishes in the fewest months.
    pick = min(results, key=lambda r: r["months"])
    # Explain the reasoning behind the choice.
    reason = "there is a deadline, so the quickest path wins"
# Otherwise, balance cost and risk by scoring each and taking the lowest combined score.
else:
    # Build a simple combined score: risk plus cost scaled down to the same ballpark.
    pick = min(results, key=lambda r: r["risk"] + r["double_run_cost"] / 5000)
    # Explain the reasoning behind the choice.
    reason = "no deadline and some risk is OK, so balance cost against risk"

# Print a blank line, then the recommendation, so it stands out.
print("\nRecommendation:", pick["name"], "->", reason + ".")
```

Sample output

```
Strategy     Months  Dbl-run$  Risk
Big Bang          6       $0   6.5
Phased           11  $15,000   3.9
Parallel          9  $12,000   2.6
Pilot             8   $9,000   3.9

Recommendation: Parallel
(risk tolerance = LOW)
```

How it works

- We describe Northern Roast: number of users, number of sites, the monthly cost of keeping the old system running, the firm's risk tolerance, and whether a deadline is squeezing us.
- STRATEGIES stores the four rollout methods. Each one carries three honest trade-offs: how long it runs, how long two systems overlap, and how risky the switch is.
- evaluate() converts those traits into three comparable numbers -- duration, the dollar cost of double-running, and a risk score that rises with more sites (more sites = more to coordinate = more risk).

PART 1.2

Managing risks & critical success factors

- The skill isn't avoiding risk - that's impossible - it's seeing it early and planning against it
- Rank each risk: Likelihood (1-5) x Impact (1-5) = a score from 1 to 25
- Sort worst-first; that ranked list is called a risk register
- A few conditions quietly carry projects to success - the Critical Success Factors (CSFs)

The instinct you already have

- How likely is it? x How bad would it be?
- A rare, minor annoyance sinks; a likely, damaging risk rises to the top.

THE RISK REGISTER

Score it, then rank it

For each risk, multiply two 1-to-5 judgements, then sort by the result:

Severity bands:

Common EIS risks

- Dirty data that won't map cleanly
- Scope creep ('just one more feature')
- User resistance (quietly using the old way)
- Too little training before go-live
- Running over budget
- Over-dependence on one key person
- Systems that refuse to talk to each other
- Going live at the worst possible time

WHAT MAKES PROJECTS SUCCEED

Critical Success Factors - inverted risks

The striking thing: almost none of these are about technology.

- Visible support from top management
- A clear, realistic scope
- A strong project manager and governance
- Deliberate change management & communication
- Real end-user training before go-live
- A solid data-quality and migration plan
- A realistic timeline & budget with contingency

Two sides of one coin

Every success factor is a risk turned inside out:

'Do end-user training' <=> 'Risk: staff under-trained at go-live.'

Manage the risks and you're practising the success factors - and the reverse.

REAL-LIFE EXAMPLE

Northern Roast builds its risk register

What it shows & how it links: Likelihood x Impact scoring in action, each risk paired with the CSF that defends it (Part 1.2).

ID	Risk (what could go wrong)	L	I	Score	Severity
R-01	Old records won't map cleanly into the new system	4	5	**20**	**Critical**
R-03	Cafe managers resist and keep their old spreadsheet	4	4	**16**	**Critical**
R-09	Go-live lands in the holiday rush	3	5	**15**	**Critical**
R-10	Leadership backs it but never shows up to decide	3	5	**15**	**Critical**
R-02	Scope creep - 'just one more report'	4	3	**12**	**High**
R-07	The one IT admin leaves mid-project	2	5	**10**	**High**

Working .xlsx ships in the package; a second tab scores CSF readiness (3 of 7 = 43%).

CODE ILLUSTRATION

Risk register scorer (1/2)

```
# Store the project's risks as a list of dictionaries -- one dictionary per risk.
# Each risk has a short id, a description, and two 1-to-5 judgements.
RISKS = [
    # A high chance and a huge impact: dirty data carried into the new system.
    {"id": "R-01", "what": "Old records don't map cleanly into the new system", "likelihood": 4, "impact": 5},
    # Common and moderately damaging: endless extra feature requests.
    {"id": "R-02", "what": "Scope creep -- 'just one more report'", "likelihood": 4, "impact": 3},
    # Likely and very damaging: staff quietly avoid the new tool.
    {"id": "R-03", "what": "Store managers resist and keep their old spreadsheet", "likelihood": 4, "impact": 4},
    # Unlikely but catastrophic: the only admin who knows the system leaves.
    {"id": "R-07", "what": "The one IT admin leaves mid-project", "likelihood": 2, "impact": 5},
    # Fairly likely and severe: going live during the holiday rush.
    {"id": "R-09", "what": "Go-live lands in the holiday rush", "likelihood": 3, "impact": 5},
    # Fairly likely and severe: leaders approve but never make decisions.
    {"id": "R-10", "what": "Leadership backs it but never shows up to decide", "likelihood": 3, "impact": 5},
]

# Make a helper that turns a numeric score into a plain-language severity label.
def severity(score):
    # A score of 15 or more (out of a maximum 25) is the top band: Critical.
    if score >= 15:
        # Return the label for the worst band.
        return "Critical"
    # A score of 9 to 14 is serious but not yet the worst: High.
    elif score >= 9:
        # Return the label for the high band.
        return "High"
    # A score of 4 to 8 is worth watching: Medium.
    elif score >= 4:
        # Return the label for the medium band.
        return "Medium"
    # Anything lower (1 to 3) is minor: Low.
    else:
        # Return the label for the least serious band.
        return "Low"
```

What it does

- Scores a list of risks (Likelihood x Impact), labels severity, and sorts worst-first
- Links to Part 1.2: the same maths as the spreadsheet, written as code

Reading the panel

Muted lines are comments (one above every line of code); light lines are the code itself.

CODE ILLUSTRATION

Risk register scorer (2/2)

```
# Walk through every risk and attach its computed score and severity label.
for risk in RISKS:
    # The score is simply how likely it is multiplied by how bad it would be.
    risk["score"] = risk["likelihood"] * risk["impact"]
    # Look up the human-readable band for that score and store it too.
    risk["severity"] = severity(risk["score"])

# Sort the whole list by score, biggest first, so priorities float to the top.
RISKS.sort(key=lambda r: r["score"], reverse=True)

# Print a heading for the prioritized register.
print("Northern Roast -- EIS risk register (worst first)\n")
# Print a table header with fixed-width columns so everything lines up.
print(f"{'ID':<6}{'Score':>6}  {'Severity':<10}{'Risk'}")
# Print a divider line under the header.
print("-" * 74)
# Loop over the now-sorted risks and print one row each.
for risk in RISKS:
    # Show id, score, severity band, and the short description of the risk.
    print(f"{risk['id']:<6}{risk['score']:>6}  {risk['severity']:<10}{risk['what']}")

# Count how many risks fell into the Critical band so we can flag them loudly.
critical_count = sum(1 for r in RISKS if r["severity"] == "Critical")

# Print a blank line, then a short action message telling the team where to start.
print(f"\nAction: {critical_count} CRITICAL risk(s) must have an owner and a plan BEFORE go-live.")
# Point specifically at the single worst risk (it is first in the sorted list).
print("Start with:", RISKS[0]["id"], "-", RISKS[0]["what"])
```

Sample output

```
ID    Score  Severity
R-01   20    Critical
R-03   16    Critical
R-09   15    Critical
R-10   15    Critical
R-02   12    High
R-07   10    High

4 CRITICAL -> owners
Start with: R-01
```

How it works

- RISKS is a list of dictionaries. Each dictionary describes one thing that could go wrong, plus two gut-feel scores from 1 to 5: how LIKELY it is and how big the IMPACT would be if it happened.
- severity() is a small "translator": give it a number and it hands back a word (Low / Medium / High / Critical), using the same cut-offs as the spreadsheet (15+, 9+, 4+, else).
- The first loop fills in each risk's score (Likelihood x Impact) and its severity word.

WATCH AFTER LESSON 1 (<= 15 MIN)

See the implementation process end to end

PRIMARY

The ERP Implementation Process Explained

youtube.com/watch?v=kV2cyb4AeH8

BACKUP (if the first link is down)

The ERP Implementation Process: 8 Key Steps

youtube.com/watch?v=_NS6KR0AJTw

Using these videos safely

- Linked, not hosted - Cyber.SoHo never downloads or re-uploads them
- Play straight from YouTube so the view (and revenue) stays with the creator
- Click-test each link before class and scan it with VirusTotal
- Third-party links can move - use the backup, or search the title

CYBER.SOHO

Lesson 2 - EIS Project Management

How the effort is run, and why the software's shape mirrors the company's

PART 2.1

EIS structure & how it mirrors the organisation

- The best mental picture (from SAP): a company is like a human body
- Separate systems - circulation, digestion, nerves - each doing its job, all connected
- An EIS is the company's nervous system: information flows instantly between the parts
- Built from modules, all sharing one database - and its shape ends up mirroring the company's

The payoff line

- A problem in one part shows up in the others - because they're wired together.
- That wiring is exactly why the org chart and the module map look alike.

BUILDING BLOCKS

Modules - one app per business area

Finance
the books & payments

HR / HCM
the people side

Sales & CRM
customers, quotes, sales

Procurement
buying things

Inventory / WMS
what's in stock & where

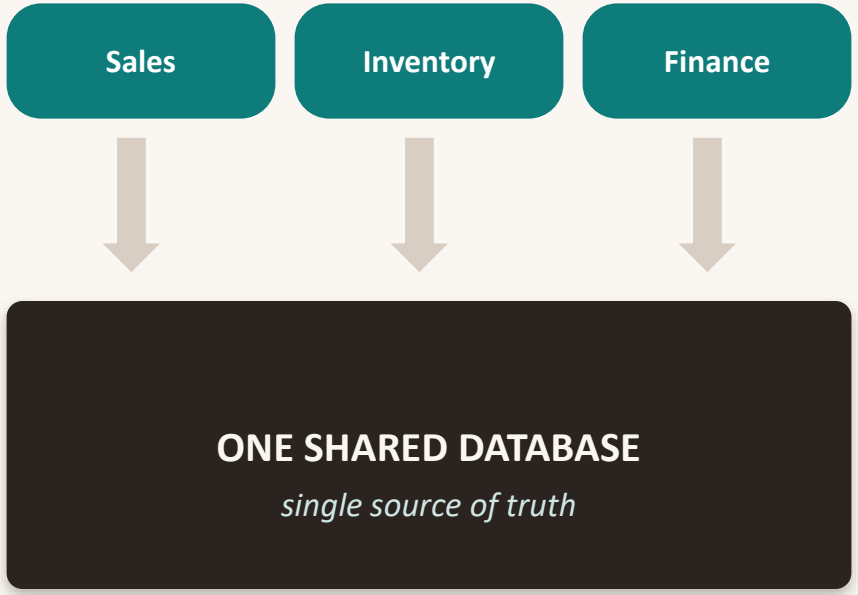
Manufacturing / MRP
what to make & when

Companies pick the modules they need and add more as they grow. (WMS = Warehouse Management System · MRP = Material Requirements Planning · HCM = Human Capital Management.)

THE MAGIC

One shared database - the single source of truth

- Every module reads from and writes to ONE shared DB (Database)
- Enter a fact once - a new customer, a price change, a received shipment - and every module sees it instantly
- That shared store is called the single source of truth, and it is the whole point
- Without it: the Northern Roast 'before' mess - one address typed three different ways, nobody sure which is right



Type it once - everyone sees it.

HOW IT IS STACKED

Three-tier architecture

screens on top

Presentation tier

The screens, app and dashboards people touch (UI)

brains in the middle

Application tier

The modules - where all the business logic lives

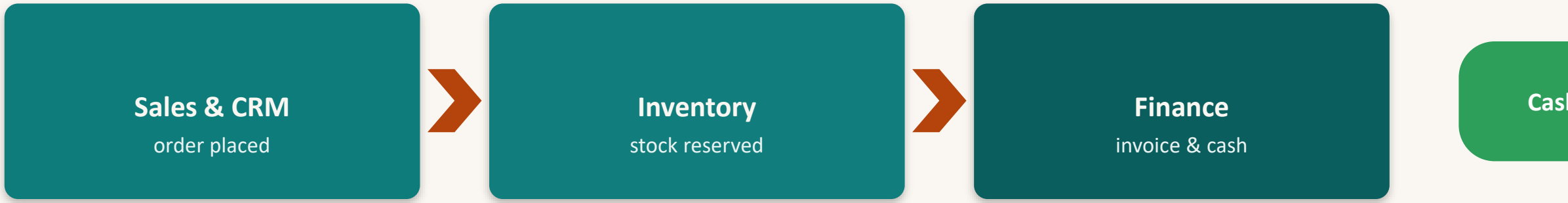
memory at the bottom

Database tier

The one shared store underneath everything

A PROCESS RUNS ACROSS MODULES

Order-to-Cash - a relay across modules

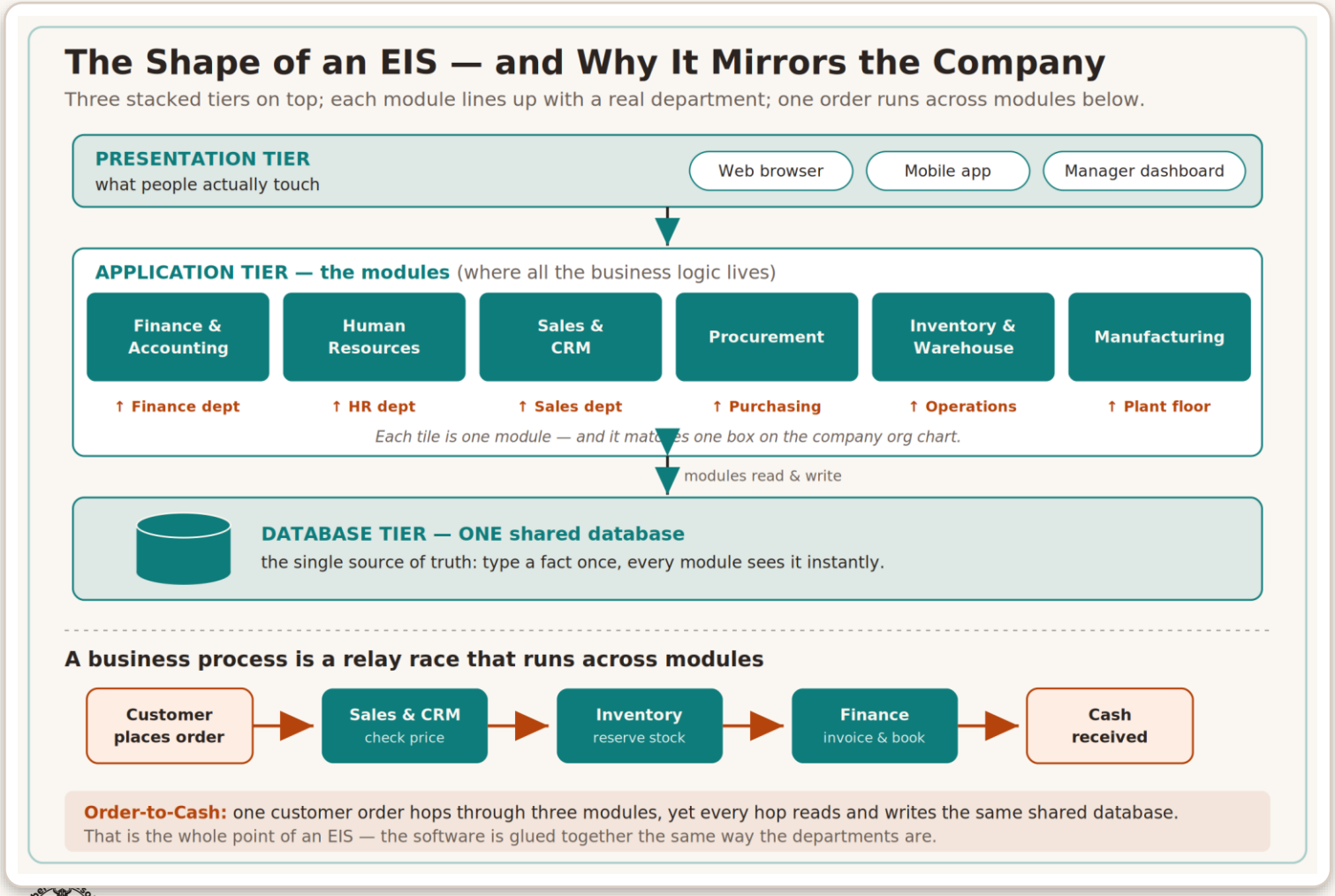


One order, three modules, one shared database - like a relay race passing a baton.

A customer places an order (Sales & CRM); the system checks and reserves stock (Inventory); an invoice goes out and the money is booked (Finance); cash arrives. The software is glued together the same way the departments are - which is why the org chart and the module map look so alike.

THE WHOLE SHAPE ON ONE PAGE

Figure 3 - EIS structure & the organisation



What to notice

- Three tiers stacked: presentation -> application -> database
- Each module tile carries a rust tag naming the department it mirrors
- The Order-to-Cash ribbon hops across Sales, Inventory and Finance
- Every hop reads and writes the same shared database

REAL-LIFE EXAMPLE

One coffee order, three modules

What it shows & how it links: modular structure + one shared database + a process running across modules (Part 2.1).

- Cafe Rue orders 20 bags: Sales logs it and bumps the customer history -> Inventory checks the shared stock and reserves 20 (now 30 left) -> Finance books the $240 of revenue.
- Not one module kept its own private copy - they all read and wrote the same shared database.
- Then Beans & Books orders 40 bags when only 30 remain - Inventory stops the relay before Finance ever books money for something Northern Roast cannot ship.
- That hand-off - one module refusing to pass the baton - is exactly the coordination an EIS exists to handle.

CODE ILLUSTRATION

Order-to-Cash module tracer (1/2)

```
# Create the "single source of truth": one dictionary that every module shares.
# stock = bags of coffee on hand; customers = who we know; ledger = money in.
DATABASE = {
    # We currently have 50 bags of house-blend beans in the warehouse.
    "stock": {"house-blend": 50},
    # We already know one wholesale customer, the corner cafe.
    "customers": {"Cafe Rue": {"orders": 0}},
    # The finance ledger starts empty -- no revenue booked yet.
    "ledger": [],
}

# --- MODULE 1: Sales & CRM -- takes the order and records the customer activity.
def sales_module(db, customer, product, qty, unit_price):
    # If this is a brand-new customer, add them to the shared customer list first.
    if customer not in db["customers"]:
        # Register the customer with a fresh order counter of zero.
        db["customers"][customer] = {"orders": 0}
    # Increase this customer's order count -- CRM keeps the relationship history.
    db["customers"][customer]["orders"] += 1
    # Work out the order's money value: quantity times the price per unit.
    order_value = qty * unit_price
    # Announce what the Sales module just did.
    print(f"[Sales & CRM] Logged order: {qty} x {product} for {customer} = ${order_value}")
    # Pass the order value forward so the next module can use it.
    return order_value

# --- MODULE 2: Inventory -- checks and reserves stock from the SAME database.
def inventory_module(db, product, qty):
    # Read how many units of this product we currently have in the shared stock.
    on_hand = db["stock"].get(product, 0)
    # If we do not have enough, stop and report the shortage.
    if on_hand < qty:
        # Tell the user the order cannot be filled and by how much we are short.
        print(f"[Inventory]   NOT ENOUGH {product}: have {on_hand}, need {qty}")
        # Signal failure so the process can halt.
        return False
    # We have enough, so subtract the ordered quantity from the shared stock.
    db["stock"][product] -= qty
    # Report the reservation and the new remaining stock level.
    print(f"[Inventory]   Reserved {qty} {product}; {db['stock'][product]} left in stock")
    # Signal success so the process can continue to Finance.
    return True
```

What it does

- Builds one shared 'database' and three modules that read & write it
- Links to Part 2.1: shows a single order crossing modules on one shared store

Reading the panel

Muted lines are comments (one above every line of code); light lines are the code itself.

CODE ILLUSTRATION

Order-to-Cash module tracer (2/2)

```
# --- MODULE 3: Finance -- books the revenue into the SAME shared ledger.
def finance_module(db, customer, amount):
    # Append a revenue record to the shared ledger list.
    db["ledger"].append({"customer": customer, "amount": amount})
    # Add up every amount in the ledger to get total revenue so far.
    total = sum(entry["amount"] for entry in db["ledger"])
    # Report what Finance booked and the running total revenue.
    print(f"[Finance]     Booked ${amount} from {customer}; total revenue now ${total}")

# --- The BUSINESS PROCESS: Order-to-Cash, running the three modules in order.
def order_to_cash(db, customer, product, qty, unit_price):
    # Print a header so each run of the process is easy to spot.
    print(f"\n--- Order-to-Cash: {customer} orders {qty} {product} ---")
    # Step 1: Sales records the order and returns its dollar value.
    value = sales_module(db, customer, product, qty, unit_price)
    # Step 2: Inventory tries to reserve the stock; remember if it succeeded.
    ok = inventory_module(db, product, qty)
    # If stock was not available, stop the whole process right here.
    if not ok:
        # Tell the user the process was aborted before any money was booked.
        print("Process halted: order not fulfilled.")
        # Leave the function early so Finance never runs for a failed order.
        return
    # Step 3: Finance books the revenue, completing the relay.
    finance_module(db, customer, value)

# Run the process once for a known customer buying 20 bags at $12 each.
order_to_cash(DATABASE, "Cafe Rue", "house-blend", 20, 12)
# Run it again for a NEW customer buying 40 bags -- watch stock and revenue update.
order_to_cash(DATABASE, "Beans & Books", "house-blend", 40, 12)

# After both orders, print the final shared state to prove all modules agreed.
print("\n--- Final shared database ---")
# Show the remaining stock; both modules changed this same number.
print("Stock left:", DATABASE["stock"])
# Show every customer and how many orders each has placed.
print("Customers :", DATABASE["customers"])
# Show total revenue booked across every order.
print("Revenue   : $" + str(sum(e["amount"] for e in DATABASE["ledger"])))
```

Sample output

```
Cafe Rue orders 20
[Sales]     $240 logged
[Inventory] reserved; 30 left
[Finance]   revenue $240

Beans & Books orders 40
[Inventory] NOT ENOUGH
            have 30, need 40
Process halted.
```

How it works

- DATABASE is one dictionary that stands in for the EIS's single shared database -- the "single source of truth". Everything else reads and writes THIS one object.
- There are three modules, each a function: sales_module, inventory_module and finance_module. Notice every one takes the same `db` and changes it. No module keeps its own private copy -- that is the whole idea of an EIS.
- order_to_cash() is the business PROCESS. It calls the three modules in sequence, passing results along like a baton in a relay race: Sales -> Inventory -> Finance.

PART 2.2

New trends in EIS provisioning

- 'Provisioning' = how you get and run it: who owns the servers, how you pay, who updates it
- This has changed more in 15 years than almost anything in business software - and it's still moving
- We'll walk the eras from oldest to newest
- The direction of travel: servers, updates and risk keep sliding from you toward the vendor

Watch this trend

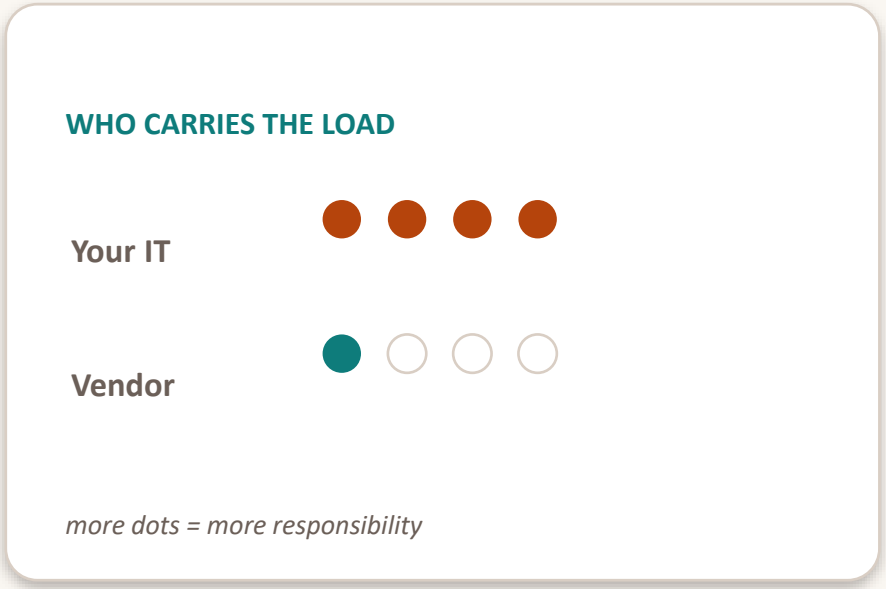
- The newest option is not automatically the best.
- You still have to do the arithmetic (the TCO) for your own company.

PROVISIONING 1

On-premises (the old way)

- Buy the software outright, buy your own servers, run it all in your building
- You pay a big one-time cost up front = CapEx (Capital Expenditure)
- Your own IT team keeps it alive and applies every update on your schedule
- You get total control - and you carry the total burden

Hosted / private cloud was the first step off your own floor: the vendor runs the servers, but you still keep your own separate copy.

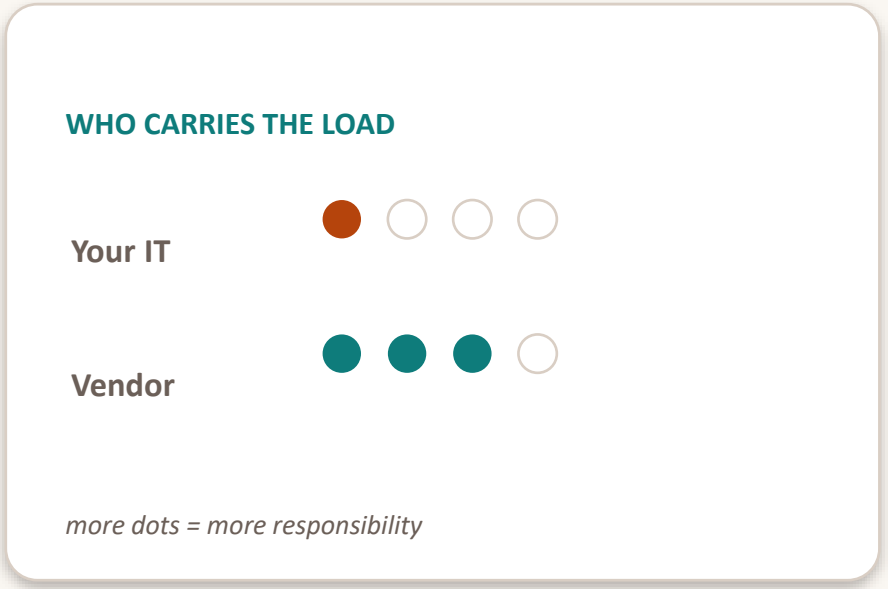


PROVISIONING 2

Multi-tenant SaaS (most new deployments)

- SaaS (Software as a Service): many customers share one instance, each firm's data walled off
- You pay a monthly per-user subscription = OpEx (Operating Expense)
- The vendor updates everyone at once - you're always current, no upgrade project
- Low up-front cost, fast to start, far less for your IT team to babysit

This is why it has taken over. Vendors like NetSuite, Salesforce and Workday were built on it.

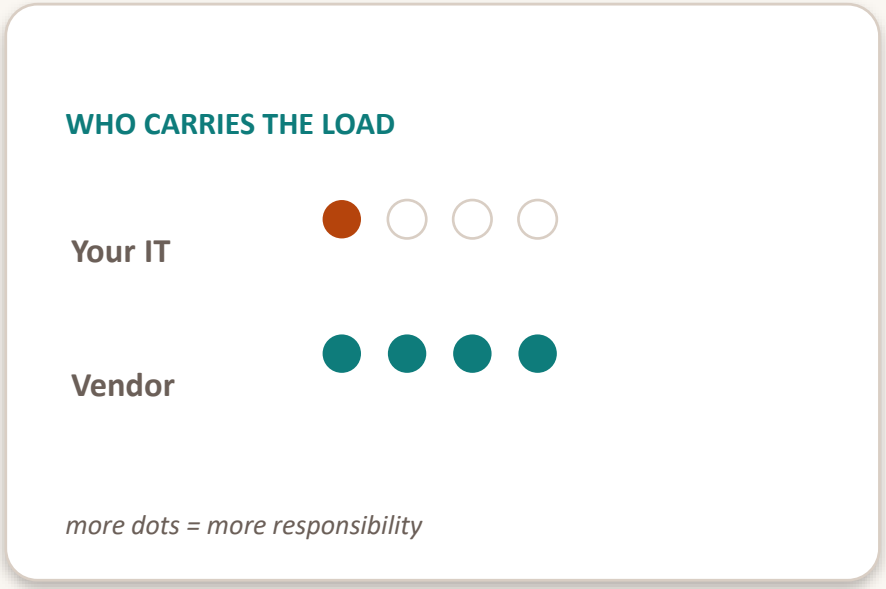


PROVISIONING 3

Composable & AI-driven (emerging)

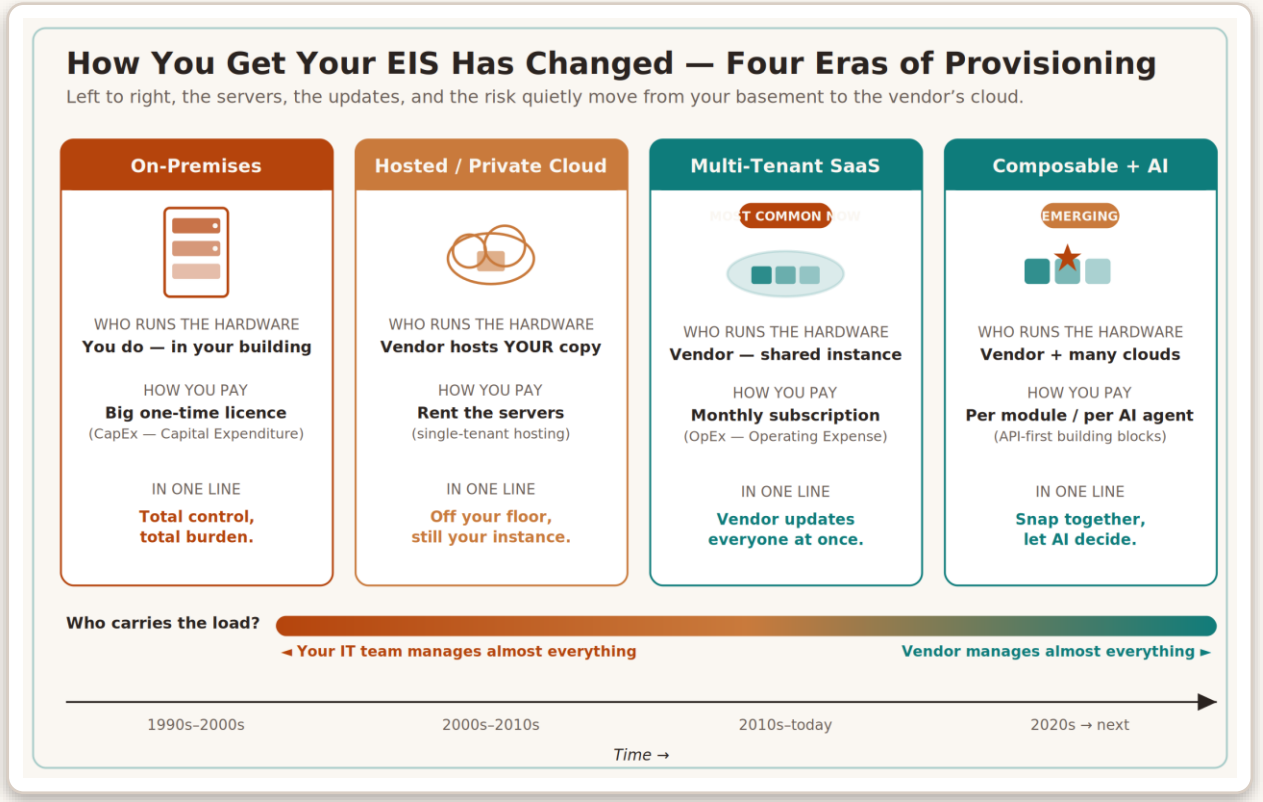
- Assemble best-fit building blocks, connected through APIs (Application Programming Interfaces)
- AI/ML woven into the modules - forecast demand, flag an odd invoice automatically
- AI agents that take small actions for you; IoT sensors for live warehouse data
- Even more of the servers, updates and risk sit with the vendor

This is the edge Gartner is watching. (AI = Artificial Intelligence · ML = Machine Learning · IoT = Internet of Things.)



HOW GETTING AN EIS HAS CHANGED

Figure 4 - The provisioning evolution



On-premises -> hosted / private cloud -> multi-tenant SaaS (most new deployments) -> composable + AI.

The gradient bar underneath is the real story: the servers, the updates and the risk keep moving from you to the vendor.

REAL-LIFE EXAMPLE

Should Northern Roast rent or buy?

What it shows & how it links: CapEx vs OpEx in real numbers - compare on 5-year TCO, not the sticker price (Part 2.2).

$263,707
Cloud (SaaS) - 5-year TCO

$650,000
On-premises - 5-year TCO

Cloud cheaper by
$386,293

- Cloud: $90/user/mo x 40 + $25k setup, +5%/yr. On-prem: $120k licence + $60k hardware + $45k setup + 20% maintenance + $55k/yr staff + $30k Year-4 refresh.
- Cloud wins from Year 1 here - on-premises makes you pay for servers and staff whether you have 40 users or 400.
- Caveat: at 400 users those fixed costs spread over ten times as many people, and the maths can flip. TCO is a calculation, not a slogan.

CODE ILLUSTRATION

Cloud-vs-On-premises TCO (1/2)

```
# --- Shared assumption: how many people will use the system.
USERS = 40
# --- How many years to compare the two options over.
YEARS = 5

# --- CLOUD (SaaS) assumptions ---
# Monthly fee charged per user by the SaaS vendor.
SAAS_FEE_PER_USER_MONTH = 90
# One-time cost to set the cloud system up (data loading, configuration).
SAAS_SETUP = 25000
# How much the subscription price rises each year, as a fraction (0.05 = 5%).
SAAS_ANNUAL_INCREASE = 0.05

# --- ON-PREMISES assumptions ---
# One-time software licence you buy outright.
ONPREM_LICENCE = 120000
# One-time cost of the servers and other hardware.
ONPREM_HARDWARE = 60000
# One-time cost to install and configure everything.
ONPREM_SETUP = 45000
# Yearly maintenance, expressed as a fraction of the licence price (0.20 = 20%).
ONPREM_MAINT_RATE = 0.20
# Yearly salary cost for the IT staff who keep it running.
ONPREM_IT_STAFF = 55000
# A hardware refresh that lands in Year 4.
ONPREM_REFRESH_YEAR4 = 30000

# Build a function that returns the CLOUD cost for a single given year number.
def cloud_cost_for_year(year):
    # The subscription grows a bit each year; year 1 has no increase yet.
    yearly_subscription = USERS * SAAS_FEE_PER_USER_MONTH * 12 * (1 + SAAS_ANNUAL_INCREASE) ** (year - 1)
    # Only year 1 carries the one-time setup fee; later years add zero setup.
    setup = SAAS_SETUP if year == 1 else 0
    # The year's total cloud cost is the subscription plus any setup.
    return yearly_subscription + setup

# Build a function that returns the ON-PREM cost for a single given year number.
def onprem_cost_for_year(year):
    # Year 1 carries all the big one-time costs: licence, hardware and setup.
    one_time = (ONPREM_LICENCE + ONPREM_HARDWARE + ONPREM_SETUP) if year == 1 else 0
    # Every year pays maintenance (a percentage of the licence) and IT staff.
    recurring = ONPREM_MAINT_RATE * ONPREM_LICENCE + ONPREM_IT_STAFF
    # Year 4 also pays for a hardware refresh; other years add zero for that.
    refresh = ONPREM_REFRESH_YEAR4 if year == 4 else 0
    # The year's total on-prem cost is the sum of those three pieces.
    return one_time + recurring + refresh
```

What it does

- Adds up 5 years of cost for cloud vs on-prem and finds the break-even year
- Links to Part 2.2: the CapEx-vs-OpEx trade-off as a running total

Reading the panel

Muted lines are comments (one above every line of code); light lines are the code itself.

CODE ILLUSTRATION

Cloud-vs-On-premises TCO (2/2)

```
# Prepare running totals (cumulative cost so far) for each option, starting at 0.
cloud_running = 0
# Start the on-prem running total at zero as well.
onprem_running = 0
# We will remember the first year cloud becomes the cheaper cumulative option.
break_even_year = None

# Print a header for the year-by-year comparison table.
print("Northern Roast -- 5-year TCO: Cloud (SaaS) vs On-premises\n")
# Print column titles, aligned in fixed-width columns.
print(f"{'Year':<6}{'Cloud (cumul.)':>18}{'On-prem (cumul.)':>20}{'Cheaper so far':>18}")
# Print a divider line under the header.
print("-" * 62)

# Loop over each year from 1 up to and including YEARS.
for year in range(1, YEARS + 1):
    # Add this year's cloud cost to the cloud running total.
    cloud_running += cloud_cost_for_year(year)
    # Add this year's on-prem cost to the on-prem running total.
    onprem_running += onprem_cost_for_year(year)
    # Decide which option is cheaper so far by comparing the running totals.
    cheaper = "Cloud" if cloud_running < onprem_running else "On-prem"
    # If cloud is cheaper and we have not recorded a break-even year yet, record it.
    if cloud_running < onprem_running and break_even_year is None:
        # Remember the first year the cloud pulls ahead on total cost.
        break_even_year = year
    # Print this year's row with both running totals rounded to whole dollars.
    print(f"{year:<6}{('$'+format(round(cloud_running), ',')):>18}{('$'+format(round(onprem_running), ',')):>20}{cheaper:>18}")

# After the loop, print a blank line, then the two 5-year grand totals.
print(f"\n5-year total  ->  Cloud: ${round(cloud_running):,}   On-prem: ${round(onprem_running):,}")
# Work out the size of the gap between the two totals.
gap = abs(round(onprem_running - cloud_running))
# Say which option wins overall and by how much.
winner = "Cloud" if cloud_running < onprem_running else "On-prem"
# Print the overall verdict.
print(f"Over {YEARS} years, {winner} is cheaper by ${gap:,}.")
# Report the break-even year, or note that one option won from the very start.
if break_even_year:
    # Tell the reader when the cloud first became the cheaper cumulative choice.
    print(f"Cloud is the cheaper running total from Year {break_even_year} onward.")
else:
    # If cloud never led, say so plainly.
    print("On-premises stayed cheaper the whole time with these numbers.")
```

Sample output

```
Yr  Cloud (cumul.)   On-prem (cumul.)  Cheaper so far
1          $68,200            $304,000           Cloud
2         $113,560            $383,000           Cloud
3         $161,188            $462,000           Cloud
4         $211,197            $571,000           Cloud
5         $263,707            $650,000           Cloud

Cloud wins from Yr 1;
cheaper by $386,293
```

How it works

- At the top we list assumptions in two groups: what the CLOUD costs (a per-user monthly fee plus a one-time setup, with a small yearly price rise) and what ON-PREM costs (big one-time licence + hardware + setup, then yearly maintenance and IT staff, plus a hardware refresh in Year 4).
- cloud_cost_for_year() and onprem_cost_for_year() each return the cost of a single year. Splitting it this way keeps the maths readable and lets us add year-specific items (like the Year-4 refresh) cleanly.
- The loop walks Year 1 to 5, adding each year's cost to a running total so we can see CUMULATIVE spend, not just one year in isolation. That is the heart of TCO: money added up over time.

WATCH AFTER LESSON 2 (<= 15 MIN)

Tie the structure & trends together

PRIMARY

ERP Systems Explained: Boost Efficiency & Integration

youtube.com/watch?v=3oDeN3EQbps

BACKUP (if the first link is down)

Enterprise Resource Planning (ERP) in 15 minutes

youtube.com/watch?v=gBXJ_PhlADQ

Using these videos safely

- Linked, not hosted - Cyber.SoHo never downloads or re-uploads them
- Play straight from YouTube so the view (and revenue) stays with the creator
- Click-test each link before class and scan it with VirusTotal
- Third-party links can move - use the backup, or search the title

WRAP-UP

The whole day, in a few sentences

- Implementation is mostly a people-and-planning problem, not a technology one
- Pick a go-live method by how much old/new overlap you'll pay for (overlap = safety, at the price of time + cost)
- Survive with a risk register (Likelihood x Impact, worst first) and invest in the CSFs (inverted risks)
- The software is modules sharing one database in three tiers; a process (Order-to-Cash) runs across them
- Buying shifted CapEx -> SaaS/OpEx -> composable + AI; the 'best' is whatever the 5-year TCO says for YOUR firm

Study offline

- HTML study companion - quiz, flashcards, live calculators
- Four commented Python programs
- Two working spreadsheets (risk + TCO)
- A five-sheet reference workbook

REFERENCES

Links used - check every one before you click

Vendors (no affiliation)

- SAP - sap.com
- Oracle NetSuite - netsuite.com
- MS Dynamics 365 - microsoft.com/dynamics-365
- Salesforce - salesforce.com
- Workday - workday.com

Bodies & standards

- PMI - pmi.org
- ISO - iso.org
- NIST - nist.gov
- The Open Group - opengroup.org
- Standish Group - standishgroup.com
- Gartner - gartner.com

Tools & videos

- Python - python.org
- VirusTotal - virustotal.com/gui/home/url
- Video 1: youtu.be/kV2cyb4AeH8
- Video 1 backup: youtu.be/_NS6KR0AJTw
- Video 2: youtu.be/3oDeN3EQbps
- Video 2 backup: youtu.be/gBXJ_PhlADQ

every external link with VirusTotal before opening it; third-party pages can move or change without notice.

REFERENCE

Glossary - every abbreviation (1 of 2)

EIS	Enterprise Information System - company-wide software on shared data
ERP	Enterprise Resource Planning - the most common kind of EIS
CRM	Customer Relationship Management - customers, leads, sales
SCM	Supply Chain Management - flow of goods, supplier to customer
HR / HCM	Human Resources / Human Capital Management - the people side
WMS	Warehouse Management System - what's in stock and where
MRP	Material Requirements Planning - what to buy and make, and when
GL	General Ledger - the master record of financial transactions
P&L	Profit and Loss - the report showing whether the business made money

IT	Information Technology - the team and systems that run computing
SaaS	Software as a Service - software you rent, run by the vendor
CapEx	Capital Expenditure - a big one-time purchase, like servers
OpEx	Operating Expense - an ongoing cost, like a subscription
TCO	Total Cost of Ownership - the full cost over time, not the sticker
ROI	Return on Investment - what you get back vs what you spent
CBA	Cost-Benefit Analysis - weighing costs against benefits
CSF	Critical Success Factor - something that must go right

REFERENCE

Glossary - every abbreviation (2 of 2)

KPI	Key Performance Indicator - a number you watch to judge progress
SOP	Standard Operating Procedure - the agreed step-by-step way
API	Application Programming Interface - the 'plug' between systems
AI	Artificial Intelligence - software that learns patterns
ML	Machine Learning - the branch of AI that learns from data
IoT	Internet of Things - devices and sensors on the internet
EDA	Event-Driven Architecture - an action triggers the next one
UI	User Interface - the screens and buttons people use

DB	Database - the organised store of the company's data
SMB / SME	Small & Medium Business / Enterprise - a smaller company
M&A	Mergers and Acquisitions - companies combining or buying
PM	Project Manager / Management - owns the plan; running projects
PMI	Project Management Institute - publishes the PMBOK guide
PMBOK	Project Management Body of Knowledge - PMI's standard
NIST	National Institute of Standards and Technology - risk framework
ISO	International Organization for Standardization - global standards

PLEASE READ IN FULL

Disclaimer, IP & Terms of Use (1 of 2)

1. Ownership & IP

All original material here - lecture, diagrams, spreadsheets, code, the interactive companion and the Northern Roast scenario - is the IP of Cyber.SoHo Educational Hub. (c) 2026. All rights reserved.

2. Permitted use

Provided for education. Personal study and classroom instruction are fine; no resale, rebranding, or passing it off as someone else's work without written permission.

3. No affiliation; trademarks

Cyber.SoHo is independent and is not affiliated with, endorsed by or sponsored by any company, body or channel named here. All trademarks belong to their owners and are used only for identification and education (nominative fair use).

4. Original, non-infringing

Written independently, paraphrasing widely known industry concepts in the author's own words. No third-party copyrighted text or images are reproduced; the concepts themselves are general knowledge.

5. Fictional example

Northern Roast Coffee Co., its staff and all figures are entirely fictional and for illustration only. Any resemblance to a real company is coincidental; the dollar amounts are teaching numbers.

6. External links & videos

Third-party links are for convenience; the author does not own or monitor them and is not responsible for their content or availability. Verify each with VirusTotal first. Videos are linked, not hosted - credit stays with the creators.

PLEASE READ IN FULL

Disclaimer, IP & Terms of Use (2 of 2)

7. No professional advice

General education only - not professional, legal, financial or project-management advice. Do not make real decisions on this basis; consult a qualified professional for your situation.

8. Software 'as is'

The code and spreadsheets are provided 'as is', without warranty of any kind. Run them at your own risk in a safe environment; review and test independently before any production use.

9. Limitation of liability

To the maximum extent permitted by law, the author is not liable for any direct, indirect, incidental or consequential damages arising from use of this material or any linked resource.

10. Ethics & integrity

Use this honestly. If you submit work, follow your institution's academic-integrity rules and cite this package. Use any security topics responsibly and lawfully.

11. Right to update

The author may correct or revise this material at any time without notice. Always work from the latest version you were given.

12. Contact

Questions about permitted use or licensing should be directed to Cyber.SoHo Educational Hub. Full terms live in the lecture document.

Thank you - questions & the quiz

Open `interactive/EIS_study_companion.html` for the 6-question self-check